

TASK 6: Machine Learning Pipeline (End-to-End)

Domain: AI / Data Science | Level: Advanced

What is this project?

An end-to-end ML pipeline in Java that loads a CSV dataset, preprocesses the data, trains a Linear Regression model from scratch (no external ML library needed), evaluates accuracy, saves the model, and makes predictions.

Key Concepts to Learn First

- CSV parsing with `BufferedReader`
- Linear Regression math: $y = mx + b$
- Train/Test split – 80% train, 20% test
- MSE (Mean Squared Error) for evaluation
- Saving model weights to a file
- Making predictions with the saved model

What is Linear Regression?

Given data points (x, y) , find the best line $y = m \cdot x + b$ that fits them. We use Gradient Descent to find m and b by minimizing the error between predictions and actual values.

Complete Java Code

```
import java.io.*;
import java.util.*;

/**
 * ML Pipeline - Linear Regression from Scratch
 * Dataset: Simple house price predictor (size -> price)
 * No external libraries needed!
 */
public class MLPipeline {

    // ■■ Hyperparameters ■■
    static final double LEARNING_RATE = 0.0001;
    static final int    EPOCHS        = 10000;
    static final double TRAIN_RATIO   = 0.8;

    public static void main(String[] args) throws IOException {

        // ■■ Step 1: Load Dataset ■■
        // CSV format: size_sqft,price
        List<double[]> data = loadCSV("house_data.csv");
        System.out.println("Loaded " + data.size() + " records.");

        // ■■ Step 2: Normalize features (scale to 0-1) ■■
        double[] sizes = data.stream().mapToDouble(r -> r[0]).toArray();
        double[] prices = data.stream().mapToDouble(r -> r[1]).toArray();
    }
}
```

```

double maxSize = Arrays.stream(sizes).max().getAsDouble();
double maxPrice = Arrays.stream(prices).max().getAsDouble();
for (int i = 0; i < sizes.length; i++) {
    sizes[i] /= maxSize;
    prices[i] /= maxPrice;
}

// ■■ Step 3: Train/Test Split ■■
int trainSize = (int)(sizes.length * TRAIN_RATIO);
double[] xTrain = Arrays.copyOfRange(sizes, 0, trainSize);
double[] yTrain = Arrays.copyOfRange(prices, 0, trainSize);
double[] xTest = Arrays.copyOfRange(sizes, trainSize, sizes.length);
double[] yTest = Arrays.copyOfRange(prices, trainSize, prices.length);

System.out.println("Training samples: " + trainSize);
System.out.println("Testing samples: " + (sizes.length - trainSize));

// ■■ Step 4: Train Linear Regression (Gradient Descent) ■■
double m = 0.0; // Slope
double b = 0.0; // Intercept
int n = xTrain.length;

for (int epoch = 0; epoch < EPOCHS; epoch++) {
    double dM = 0, dB = 0;

    // Calculate gradients
    for (int i = 0; i < n; i++) {
        double predicted = m * xTrain[i] + b;
        double error = predicted - yTrain[i];
        dM += error * xTrain[i];
        dB += error;
    }

    // Update weights
    m -= (LEARNING_RATE * dM / n);
    b -= (LEARNING_RATE * dB / n);

    // Print progress every 1000 epochs
    if (epoch % 1000 == 0) {
        double mse = calculateMSE(xTrain, yTrain, m, b);
        System.out.printf("Epoch %5d | MSE: %.6f\n", epoch, mse);
    }
}

// ■■ Step 5: Evaluate on Test Data ■■
double testMSE = calculateMSE(xTest, yTest, m, b);
System.out.printf("\nFinal Test MSE: %.6f\n", testMSE);
System.out.printf("Model: price = %.4f * size + %.4f (normalized)\n", m, b);

// ■■ Step 6: Save Model ■■
saveModel(m, b, maxSize, maxPrice);
System.out.println("Model saved to model.txt");

// ■■ Step 7: Make a Prediction ■■
double inputSize = 1500; // square feet
double predicted = predict(inputSize, m, b, maxSize, maxPrice);
System.out.printf("\nPrediction: A %.0f sqft house costs approx. $%.2f\n",
    inputSize, predicted);
}

```

```

// ■■ Load CSV file ■■
static List<double[]> loadCSV(String filename) throws IOException {
    List<double[]> data = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
        String line;
        br.readLine(); // Skip header
        while ((line = br.readLine()) != null) {
            String[] parts = line.split(",");
            double x = Double.parseDouble(parts[0].trim());
            double y = Double.parseDouble(parts[1].trim());
            data.add(new double[]{x, y});
        }
    }
    return data;
}

// ■■ Calculate Mean Squared Error ■■
static double calculateMSE(double[] x, double[] y, double m, double b) {
    double sum = 0;
    for (int i = 0; i < x.length; i++) {
        double diff = (m * x[i] + b) - y[i];
        sum += diff * diff;
    }
    return sum / x.length;
}

// ■■ Save model to file ■■
static void saveModel(double m, double b,
    double maxSize, double maxPrice) throws IOException {
    try (PrintWriter pw = new PrintWriter("model.txt")) {
        pw.println("slope=" + m);
        pw.println("intercept=" + b);
        pw.println("maxSize=" + maxSize);
        pw.println("maxPrice=" + maxPrice);
    }
}

// ■■ Make prediction (denormalized) ■■
static double predict(double sizeSqft, double m, double b,
    double maxSize, double maxPrice) {
    double normalizedSize = sizeSqft / maxSize;
    double normalizedPrice = m * normalizedSize + b;
    return normalizedPrice * maxPrice; // Denormalize
}
}

```

Sample Dataset (house_data.csv)

```

size_sqft,price
500,150000
750,200000
1000,250000
1200,300000
1500,380000
1800,450000
2000,500000
2500,620000
3000,750000
3500,900000

```

Expected Output

```
Loaded 10 records.
Training samples: 8
Testing samples: 2
Epoch    0 | MSE: 0.098234
Epoch 1000 | MSE: 0.003421
Epoch 5000 | MSE: 0.000123
Epoch 9000 | MSE: 0.000041

Final Test MSE: 0.000038
Model: price = 0.8921 * size + 0.0231 (normalized)
Model saved to model.txt

Prediction: A 1500 sqft house costs approx. $381500.00
```

How to Run

```
# 1. Create house_data.csv with data above
# 2. Compile:
    javac MLPipeline.java

# 3. Run:
    java MLPipeline
```

TIP: You'll learn Linear Regression • Gradient Descent • Train/Test split • MSE evaluation • Model saving